



SMART Connect API Interface Developer Guide 2024 R1



SMART Connect API Interface Developer Guide 2024 R1

© 2024 Soprano Design Pty Ltd and Confidentiality Statement

All rights reserved Soprano Design Ltd. No parts of this work may be reproduced in any form or by any means - graphic, electronic, or mechanical, including photocopying, recording, taping, or information storage and retrieval systems - without the written permission of SOPRANO Design Ltd.

Products that are referred to in this document may be either trademarks and/or registered trademarks of the respective owners.

While every precaution has been taken in the preparation of this document, the publisher and the author assume no responsibility for errors or omissions, or for damages resulting from the use of information contained in this document or from the use of programs and source code that may accompany it. In no event shall the publisher and the author be liable for any loss of profit or any other commercial damage caused or alleged to have been caused directly or indirectly by this document.

Printed: March 2024

Table of Contents

Introduction	5
Muti-Channel Message Delivery	6
Low or No Code Integration	7
Getting Started	8
RESTful Architechture	8
Base URL Endpoint	8
URL Endpoint Security	8
Authentication	9
Connect API Real-Time Mode	11
Message Endpoint	11
Sending Messages	11
Mandatory Request Body	12
Optional Request Body Details	13
Example Connect API Requests	26
SMS Request	26
Email Request	27
RCS Request	27
WhatsApp Request	29
GBM Request	32
Push Notifications	34
Receiving Messages	35
Creating Templates	38
Connect API HTTP Batch Mode	40
Connect API SFTP Batch Mode	42
Create Lists in The Portal	44
Fraud Detection and Prevention within Connect API	45
Introduction	45
Fraud Detection Setup in MEMS	46
Fraud Detection Parameters	49
Advantages of Fruad Check and Prevention within Connect API	51
Appendix	51

How to get access token	51
Connect API Status and Error Codes	52
WhatsApp set Message as Read and MO Parameters	53

Introduction

Connect API has been built by developers, for developers, to enable low-code or no-code integration. This makes building interactive communications into User's application very user-friendly.

Connect API offers the following :

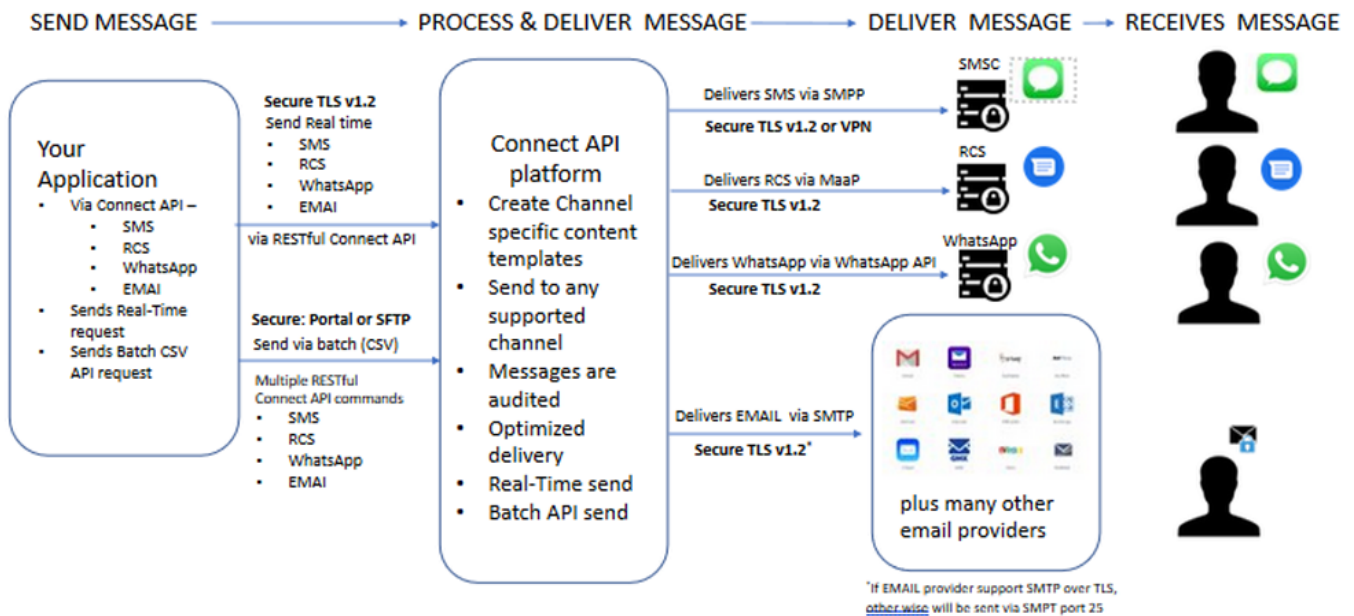
- Multi-channel messaging - SMS, RCS, WhatsApp and EMAIL
- Secure message delivery
- RESTful API
- Enables low-code integration
- Message Template - Channel specific content is built in the portal (no code)
- Real-Time API message mode
- Batch API message mode enables communicating to multiple channels from a single CSV file
- Interactive 2-way messaging support enables response to be managed by Inbound Service

Note: To avoid any issues, please coordinate with Soprano Customer Service or Sales team if you're updating or changing your SSL certificate.

Multi-Channel Message Delivery

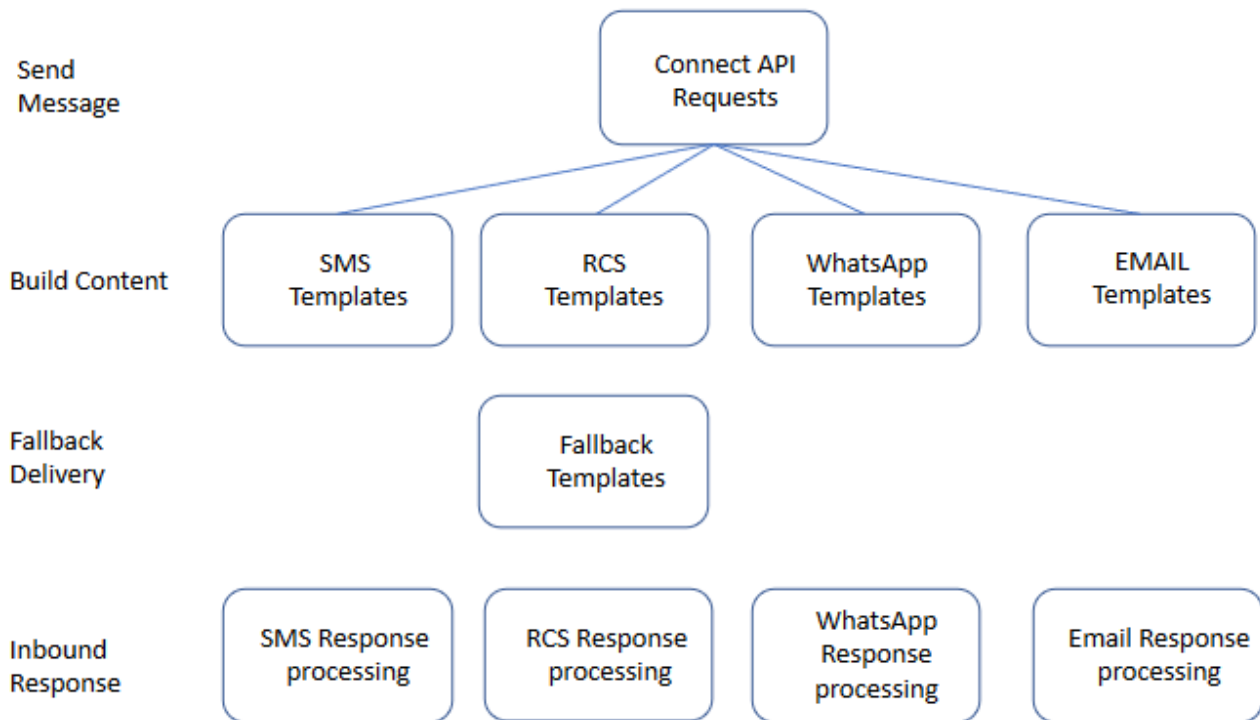
Building a messaging solution across multiple channels is complex. The Connect API handles the multi-channel complexity using a single API.

Connect API multichannel delivery



Low or No Code Integration

The most powerful capability of the Connect API is how it enables low code integration since the Connect API only has to focus on sending the message. The message content and message response creation and processing is handled via the portal. No code is required to create and manage content. This makes it easy for developers to integrate since the complexity is negated.



1. Send Message - Connect API requests to a specified channel. For Example ; RCS, RCS Template, Fallback Template and Inbound Response Order.
2. Content for the supported channels are created in the portal and saved as a template. The template can then be set in the Connect API request.
3. Fallback capability is available for RCS and WhatsApp message types only; it is not applicable to SMS message type and not available to EMAIL message type.
4. For example, in an RCS template, select Fallback Channel as SMS, and an SMS Template as a Fallback Template, to be triggered if an RCS message can't be delivered to mobile handset. It will automatically fallback to an SMS Template chosen that will deliver an SMS message content to ensure delivery to all recipients.
5. Use the Inbound Service in the portal to process customer responses to enable a conversation with the recipient. This can be done by configuring an Inbound order on the portal and using the Inbound Order ID in the replyTo parameter. Currently, replyTo Inbound is supported for SMS, RCS and WhatsApp message types and is not applicable to EMAIL message type.

Getting Started

The Connect API requires a license for each channel used. Add new channels via the standard order process. Once processed, the channel will be added to the Customer account by the Customer Service team. The Customer Support team will reach out when it has been activated. This is normally completed within 2 working days.

RESTful Architecture

Connect API provides a platform-agnostic interface that the application client can utilise which is independent of the environment or programming language being used.

Connect API is built on a **REST** (**R**epresentational **S**tate **T**ransfer) which is the foundation of the World Wide Web and is widely used for API development. Connect API is considered 'RESTful' as it conforms to a REST architecture.

All Connect API requests are sent over HTTPS via JSON payloads and available through a secure and authenticated URL.

Base URL Endpoint

All API parameters referenced in this document start with a base URL endpoint.

enterprise.messagingsuite.smart.com.ph/cgpapi/

URL Endpoint Security

SMART Messaging only supports HTTPS requests, secured in transit using TLS v1.2.

Authentication

Using Connect API's real-time mode, can authenticate in the following ways:

API Key

Our recommendation is using the Authentication with API key, where the following two headers are expected in the request:

Name	Value
X-MEMS-API-ID	<API ID>
X-MEMS-API-KEY	<API Key>

Sample send SMS usage for API Key

```
POST enterprise.messagingsuite.smart.com.ph/cgpapi/messages/sms
X-MEMS-API-ID: 12345678
X-MEMS-API-KEY: abcdefgh1234567890abcdefgh1234567890
Content-Type: application/json;charset=UTF-8
Content-Length: 358
{
  "messageType": "sms",
  "destination": "61299002200",
  "text": "Hello Connect API"
}
```

OAuth2

Authentication Header including OAuth2 is standard and most secure option where user can have an access token from separate endpoint. Following are the details:

- The access token returned in response will expire within 30 minutes in the same response
- This acts as both a resource and an authorization server
- There is no automatic retrieval and a new token has to be created after the token expires
- For more details, please see the official OAuth 2.0 specification.

Sample request:

```
POST /cgpapi/auth/token
content-length: 191
content-type: application/x-www-form-urlencoded
grant_type=client_credentials&scope=messages&client_id=123456789&client_secret=abcdefgh1234567890abcdefgh1234567890
```

Sample response:

```
Content-Type: application/json
Content-Length: 1741
Cache-Control: no-store
Pragma: no-cache
{
  "access_token": "xxxx.yyyy.zzzz",
  "token_type": "Bearer",
  "expires_in": 1800
}
```

Sample send SMS usage for OAuth2

```
POST enterprise.messagingsuite.smart.com.ph/cgpapi/messages/sms
Authorization: Bearer <access_token>
Content-Type: application/json; charset=UTF-8
Content-Length: 358
{
  "messageType": "sms",
  "destination": "61299002200",
  "text": "Hello Connect API"
}
```

Basic Authentication

Another authentication method is using the Basic Authentication in the HTTP header.

Authorization: Basic {base64-encoded username:password}.

Sample send SMS usage with Basic Authentication:

```
POST enterprise.messagingsuite.smart.com.ph/cgpapi/messages/sms
Authorization: Basic abcdefl234567890==
Content-Type: application/json; charset=UTF-8
Content-Length: 358
{
  "messageType": "sms",
  "destination": "61299002200",
  "text": "Hello Connect API"
}
```

Soprano OAuth2 (Legacy Authentication)

The authentication method being used by existing customers.

The following samples describe the Soprano Legacy OAuth 2 authentication request and response:

To get new Access Token:

Sample request:

```
POST /cgpapi/auth/login
content-length: 191
content-type: application/json
{
  "username": "abc.efg@example.com",
  "password": "xxxxxx"
}
```

Sample response:

```
Content-Type: application/json
Content-Length: 1741
{
  "accessToken": "abc.123.dummydata",
  "expiresIn": 1800,
  "tokenType": "Bearer",
  "refreshToken": "xyz.345.dummydata",
  "grantType": "password"
}
```

To extend the Access Token:

Sample request:

```
POST /cgpapi/auth/refresh
content-length: 191
```

```
content-type: application/json
Authorization: Bearer <accessToken>
{
  "refreshToken": "xyz.345.dummydata"
}
```

Sample response:

```
Content-Type: application/json
Content-Length: 1741
{
  "accessToken": "def.456.dummydata",
  "expiresIn": 1800,
  "tokenType": "Bearer",
  "refreshToken": "xyz.345.dummydata",
  "grantType": "password"
}
```

Sample send SMS usage for Soprano Legacy

```
POST enterprise.messagingsuite.smart.com.ph/cgpapi/messages/sms
Authorization: Bearer <accessToken>
Content-Type: application/json; charset=UTF-8
Content-Length: 358
{
  "messageType": "sms",
  "destination": "61299002200",
  "text": "Hello Connect API"
}
```

Connect API Real-Time Mode

Connect API implements a real-time and no code methodology. All the content associated by any of the messaging channels you use like SMS, RCS, WhatsApp or EMAIL is created and maintained in the portal (requires no code).

Message Endpoint

POST enterprise.messagingsuite.smart.com.ph/cgpapi/messages/:messageType

Path parameter - String enum [sms, whatsapp, rcs, email]

Sending Messages

Set the HTTP header to `Content-Type: application/json; charset=UTF-8`

Mandatory Request Body

Parameter	Description
messageType	String Enum [sms,whatsapp,rcs,email]
destination	String Mobile Numbers- Valid MSIDN E.164 format (recipients' mobile number international format) or Valid email address - format e.g. recipient.name@domain.com or Valid device IDs (for push notifications) Note: MSISDN - Max 15 digits Email address only valid if messageType=email In real time mode, it may contain multiple comma separated destinations up to a max of 250 destinations.
text (conditional)	String Text content to be delivered to recipient. Note: If templateName is not provided, then text must be provided. If both text and templateName are provided then text will be ignored. MessageType=sms Max 1400 characters (Note: If exceeding 1400 characters the message gets trimmed) MessageType=rcs Max 4000 characters MessageType=whatsapp Max 4000 characters (or maximum 1Mb in size) MessageType=email Max 10000 characters (or maximum 1Mb in size)
Body (conditional)	String Text content of push notifications delivered to the recipient.
templateName (conditional)	String Name of a previously provisioned message template Note: Max length ; 100 characters. Templates can support mailmerge variables either as {Balance} or {1} is a key in keyValues or can be a column header title in a CSV file(in batch mode only). Templates can be created for SMS, RCS, WhatsApp and EMAIL.

Optional Request Body Details

Parameter	Description
keyValues	String Values to mailmerge into templateName or text Format is key value pairs separated by semicolon e.g. For merge field type {name} where name could be 'Your Balance is {Balance}' Balance=99;day=Monday;CardType=Credit Card For merge field type {n} where 'n' could be 'Your Balance is {1}' 1=99,2=Monday,3=Credit Card Note: Max Length 2000 characters
clientMessageId	String Your message reference e.g. Card:4324, added to message audit record and can be tagged to reply messages. Note: Max Length 128 characters
costCentre	String Cost Centre to associate sent message. Can be sent per message. Useful for cross billing message costs. If not set, user's Cost Centre Label from the messaging platform (if defined) will be used. Note: Max Length 100 characters
replyToTON	Integer enum [0,1,3,8,10] means [Inbox,mobile,inbound,email,URL] Type of reply address used to forward user replies and delivery receipts. replyToTon=0 (Inbox) this is the default option. The reply will be returned to the portal Inbox, message data is always recorded in the portal regardless of the replyToTon setting. replyToTon=1 (mobile) will return response back to a mobile number specified in the replyTo parameter, this option is rarely used. replyToTon=3 (inbound) will send response to the Inbound order specified in the replyTo parameter replyToTon=8 (email) will return response back to an email address specified in the replyTo parameter. replyToTon=8 (email) can be used to set a different return path from the sender if messageType=email Note: If message type is email then replyToTON supports only values 0 or 8. replyToTon=10 is URL will return response back to a HTTPS URL specified in the replyTo parameter.

Parameter	Description
replyTo	String Inbound requires the Inbound Order ID number e.g replyTo=1363621. This can be found in the portal Inbound. Select the Inbound Order to have response sent to and see the Inbound Order ID on the Summary page. ReplyTo address related to replyToTon setting [mobile, inbound,email,URL]
source	String The service Number/ Service Identity to use as the sender ID / originator as associated with your account. Valid source types are: • MSISDN (short code or long number) • AlphaTag/Sender ID • Email address (From address) Note: Max Length 50 characters Source can be set as either: Pool address which are able to be tracked including replies. An account can have multiple pool addresses assigned to it. Non-Pool Address which are not tracked and used when sent to a specific address. An account can have multiple non-pool addresses assigned to it. If specifying a non-pool address the message tracking features are disabled including the linking of clientMessageId of the submitted message, and replyToTon and ReplyTo is ignored.
destinationTon	Integer Enum [1,2] means [International,National] Specify 1 if you want set the destination MSISDN format to international E.164 format. Specify 2 if you want only use domestic numbers. Not providing a destinationTON value (default) number can fit into multiple countries. User's MSISDN Conversation preference is used and this is set in the portal.
launchTimestamp	String date-time Message scheduled time in absolute format ISO-8601. Format as defined in https://www.w3.org/TR/NOTEdatetime e.g. 2020-070-21T17:20:00Z+01:00 If launchTimestamp is not set, the message will be sent immediately. Note: Not applicable to Email. Email can only have an immediate delivery date and cannot be scheduled. For launchTimestamp or delay (almost the same thing) the values are expected to be an absolute or relative value. Meaning the value must be at least 5-6 minutes in advance otherwise it is ignored and sent immediately.

Parameter	Description
validityTimestamp	String date-time Message scheduled time in absolute format ISO-8601. Format as defined in https://www.w3.org/TR/NOTEdatetime e.g. 2020-070-21T17:20:00Z+01:00
sourcePresentation	String For email display name, it overrides the name set for the source address or the display name set in the template. Note: Max Length 2000 characters.
subject	String Subject is text field to describe the subject of the email, and overrides the "Subject" field of the template if used in API. Note: Max Length 2000 characters.
delay (conditional)	Integer Scheduled Message format is relative to number of minutes to delay sending the message. Supports either absolute or relative format. If both provided, absolute is taken, and relative is ignored. Note: Not applicable to Email. Email can only have an immediate delivery date and can't be scheduled.
valid (conditional)	Integer Message validity is in relative format as number of minutes. Supports either absolute or relative format. If both provided, absolute is taken and relative is ignored.
registered	Integer enum [0,1,2,3,4,5] means [off,registered>Error DRs,Send intermediary DR only. Send both intermediary and network DRs. This is reserved for Error only DRs. Note: Applicable to messageType=SMS only. Other messageType are set to registered by default.
username	String Only applicable in real-time mode, if using username and password authentication.
password	String Only applicable in real-time mode, if using username and password authentication
fallbackMessageType	String Enum [SMS] Currently only applicable when messageType=RCS. Can currently only select SMS as a fallback method. Note: It is also possible to set the Fallback Channel in the RCS order. If set in the order no need to specify it here. Currently supported for message type RCS or WhatsApp. If specifying FallbackMessageType / FallbackTemplateName here, then these values take precedence over the values in the order message.

fallbackTemplateName	String The name of previously provisioned template of provided fallbackMessageType Note: Max Length 100 Characters. It is also possible to set the Fallback TemplateName in the RCS order. If set in the order, no need to specify it here. Currently supported for message type RCS or WhatsApp. If specifying FallbackMessageType / FallbackTemplateName here, then these values take precedence over the values in the order message.
fraudcheck	fraudCheck is an optional value of boolean nature. Possible values may be true or false. The default value being false. Note: Billing for Fraud Check is done in two phases. One billing record to check if Fraud is detected (or not detected). The second is when the message is sent (or not sent).
fraudRiskThreshold	Default values are as on Fraud (SMART Messaging) preference page. Refer to Online help - Company Setup sections. fraudRiskThreshold is an optional parameter.
whatsapp (conditional, applicable for messageType = whatsapp)	Object (includes objects and parameters within), the only way to utilize new WhatsApp message types (Interactive, Location, contact, reaction, media (image, video, document, and audio) and WhatsApp Templates (pre-defined templates on Meta business manager) For more details on new features supported and Samples check the WhatsApp Section in Appendix
type	String Mandatory within “whatsapp” object Determines the type of WhatsApp message to be sent. Values: [text] [image] [video] [document] [audio] [sticker] [template] [interactive] [location] [contacts] [reaction]
text	Object Conditional within “whatsapp” object when type = “text” Includes below parameters: - body: String Mandatory, represents body of the text message Note: Max length 4096 characters The text can contain URLs that begin with http:// or https:// and formatting. See available formatting options here. If there is a URL in the text and the user wants to include a preview box in text messages (preview_url: true), URL included must start with http:// or https:// -https:// URLs are preferred. Hostname must be included since IP addresses are not matched. - preview_url: Boolean Optional within “text” Note: Default: false. By default, WhatsApp recognizes URLs and makes them clickable, a preview box can be included with more information about the link. Set this field to true to include a URL preview box.

	Most of the time, the receiver will see a URL they can click on when you send an URL, set <code>preview_url</code> to true, and provide a body object with a http or https link. URL previews are only rendered after one of the following has happened: • The business has sent a message template to the user. • The user initiates a conversation with a "click to chat" link. • The user adds the business phone number to their address book and initiates a conversation.
image	Object Conditional within whatsapp object when type= "image". Includes the below parameters: • <code>url</code>: String Mandatory within "image" object, represents the link to the image to be shared with WhatsApp receiver. Supports HTTP/HTTPS URLs. • <code>caption</code>: String Optional within image object. Describes the specified image for WhatsApp message receiver
document	Object Conditional within whatsapp object when type="document", includes below parameters: • <code>url</code>: String Mandatory within document object, represents the link to the document to be shared with Whatsapp receiver. Supports HTTP/HTTPS URLs. • <code>caption</code>: String Optional within "document" object. Represents a description tp specified document for WhatsApp message receiver
video	Object Conditional within "whatsapp" object when type="video" Includes below parameters: • <code>url</code>: String Mandatory, represents the link to the video to be shared with WhatsApp receiver. Supports HTTP/HTTPS URLs. • <code>caption</code>: String Optional. Describes the specified video for WhatsApp message receiver
audio	Object Conditional within whatsapp object when type="audio" Includes below parameters:

	• <code>url</code>: String Mandatory within audio object, represents the link to the audio to be shared with WhatsApp receiver. Supports HTTP/HTTPS URLs. • <code>caption</code>: String Optional within audio object. Describes the specified audio for WhatsApp message receiver
<code>interactive</code>	Object Conditional within whatsapp object when <code>type="interactive"</code> The components of each interactive message generally follow a consistent pattern: <code>header</code>, <code>body</code>, <code>footer</code>, and <code>action</code> (button or list).
<code>type</code>	String Mandatory within interactive object Values: <code>[button]</code>, <code>[list]</code> Mandatory for the when <code>type = "interactive"</code>. To determine action type that will be required from the WhatsApp message receiver.
<code>header</code>	Object Optional within the <code>interactive</code> object, for the header content, Header can include video, image, document, or text. Fields include: • <code>type</code>: String. Required within header object, represents header type to be used within the interactive message. Supported values: - <code>text</code>: Used for List Messages, Reply Buttons - <code>video</code>: Used for Reply Buttons. - <code>image</code>: Used for Reply Buttons. - <code>document</code>: Used for Reply Buttons. • <code>text</code>: String Conditional within header when <code>type = "text"</code>. Represents text for the header for the WhatsApp message. • <code>image</code>: Object Conditional within header object when <code>type= "image"</code>. - <code>url</code>: String Mandatory within image object, represents the link to the image to be shared with WhatsApp receiver as part of interactive header. Use only with HTTP/HTTPS URLs.

	- <code>caption</code>: String Optional within image object. Describes the specified image for WhatsApp message receiver within interactive message header • <code>document</code>: Object Conditional within header object when <code>type</code>="document" in interactive message header. Fields include: - <code>url</code>: String Mandatory within document object, represents the link to the document to be shared with WhatsApp receiver as part of interactive message header. Use only with HTTP/HTTPS URLs. - <code>caption</code>: String Optional within document object within header of an interactive message. Describes the specified document for WhatsApp message receiver within interactive message header • <code>video</code>: Object Conditional within header object when <code>type</code>="video" for an interactive message - <code>url</code>: String Mandatory within video object, represents the link to the video to be shared with WhatsApp receiver within an interactive message header. HTTP/HTTPS URLs only. - <code>caption</code>: String Optional within <code>video</code> object Describes the specified video for WhatsApp message receiver represents the link to the video to be shared with WhatsApp receiver within an interactive message header
<code>body</code>	Object Mandatory within <code>interactive</code> object, it contains the following field: <code>text</code>: String Mandatory Represents the content of the message. Emojis and markdown are supported. Maximum length: 1024 characters
<code>footer</code>	Object Optional within <code>interactive</code> object, it contains the following field: <code>text</code>: String Mandatory Represents the content of the message. Emojis and markdown are supported. Maximum length: 60 characters

action	Object Mandatory within <code>interactive</code> object Represents content on which action to be provided by WhatsApp message receiver (list of items, or buttons). It includes the following fields: • <code>name</code>: String Conditional, within an action object, only when <code>type = "list"</code> within an <code>interactive</code> object. Represents list name displayed for WhatsApp message receiver. • <code>sections</code>: Array of objects, Conditional, within an action object when <code>type = "list"</code> within <code>interactive</code> object. Represents the sections of items within the list displayed for the WhatsApp message receiver. Each section includes an object array - <code>rows</code> - and title (optional string), maximum of 10 sections within one message. Fields include: - <code>title</code>: String Conditional within sections array, only when it includes more than one section. - <code>rows</code>: Array of objects Mandatory within <code>sections</code> object, at least one row object and maximum 10. Each row represents list item for the WhatsApp receiver to select from, each row item. Includes below fields: ▪ <code>title</code>: String Mandatory, represents list item title displayed for WhatsApp receiver, Maximum length 24 characters. ▪ <code>postbackData</code>: String Mandatory, represents unique identifier to be returned on the webhook URL, when WhatsApp message receiver selects a menu item. Maximum length: 72 characters ▪ <code>description</code>: String Optional, for further describing a given list item as viewed by WhatsApp message receiver
---------------	--

	• <code>buttons</code> : Array of objects Conditional, at least 1 must be present within an <code>action</code> object when <code>type = "button"</code> within <code>interactive</code> object. Each button object includes the below parameters: • <code>type</code>: String Mandatory, only allowed value is <code>"reply"</code> • <code>title</code>: String Mandatory and should be unique within the message. Represents the content displayed on the button as viewed by. Emojis are supported, markdown is not. Maximum length: 20 characters. • <code>postbackData</code>: String Mandatory. Must be set as unique value identifier for the button, where set value is returned as message on webhook URL (not displayed to WhatsApp receiver), whenever the button is clicked by the message receiver. Maximum length: 256 characters.
<code>location</code>	Object Conditional within <code>whatsapp</code> object when <code>type = "location"</code> Includes the below Parameters: ▪ <code>longitude</code>: Double Mandatory, represents longitude in degrees for the location viewed by WhatsApp message receiver. Ex: 151.20982 ▪ <code>latitude</code>: Double Mandatory, represents latitude in degrees for the location viewed by WhatsApp message receiver. Ex: -33.83853 ▪ <code>name</code>: String Optional describes the name of location to be viewed by the receiver. Ex: North Sydney ▪ <code>address</code>: String Optional, represents address for location, will be only displayed if name field is present. Maximum 512 characters.
<code>reaction</code>	Object Conditional within <code>whatsapp</code> object when <code>type = "reaction"</code>. For reacting on a message in the conversation. Includes the following fields: - . Includes below parameters: - o <code>messageId</code>: String. Id for the message to which set a reaction - o <code>emoji</code>: String. The emoji used by receiver. Ex: 👍

contacts	Object Conditional within <code>whatsapp</code> object when <code>type= "contacts"</code> Includes the below Parameters: ○ <code>addresses</code> an Array of Objects ○ Optional within the <code>"contacts"</code> object. Holds the address(es) of the contact being shared on whatsapp. Includes the below fields: ▪ <code>type</code>: String. Optional describes the type of address. ex: Work. Maximum length: 100 ▪ <code>street</code>: String. Optional, describes the unit, building number and street name. ex: 1 Blue Street. Maximum length: 100 ▪ <code>city</code>: String. Optional describes the city name. ex :North Sydney. Maximum length: 100 ▪ <code>state</code>: String. Optional . Represents state abbreviation. Ex:NSW. Maximum length: 100. ▪ <code>zip</code>: String. Optional ZIP code. Ex "2060". Maximum length: 20 ▪ <code>country</code>: String. Optional. Full co:untry name "Australia". Maximum length: 100 ▪ <code>countryCode</code>: String. Optional. 2 letter abbreviation. Ex:AU ○ <code>birthday</code>: String. Optional YYYY-MM-DD formatted string. Maximum length: 10 ○ <code>emails</code>: Array of objects. Optional. Includes below parameters: ▪ <code>email</code>: String. Optional. Email address. ▪ <code>type</code>: String. Optional. Standard values are HOME and WORK. ○ <code>name</code>: Object. Mandatory. Includes below parameters: ▪ <code>formattedName</code>: String. Mandatory. Full name. Maximum length: 500 ▪ <code>firstName</code>: String. Optional. First name. Maximum length: 100 ▪ <code>lastName</code>: String. Optional. Last name. Maximum length: 100 ▪ <code>middleName</code>: String. Optional. Middle name. Maximum length: 100 ▪ <code>suffix</code>: String.
----------	---

	Optional. Name suffix. Maximum length: 100 ▪ prefix: String. Optional. Name prefix. Maximum length: 100 o org: Object Optional. Contact organization information formatted as an org object. The object can contain the following field: Contact organization information formatted as an org object. The object can contain the following fields: ▪ title: String. Optional. Contact's business title. Maximum length: 100 ▪ department: String. Optional. Department name. Maximum length: 100 ▪ company: String. Optional. Name of the contact's company. Maximum length: 100 • phones: Array of Objects. Optional. Contact phone number(s) formatted as a phone object. The object can contain the following fields: - type: string. Optional. Automatically populated with the `wa_id` value as a formatted phone number. - phone: String Optional. Standard Values are CELL, MAIN, IPHONE, HOME, and WORK. - waId: string Optional. WhatsApp ID. o urls Array of objects. Contact URL(s) formatted as a urls object. The object can contain the following fields: - type: String - Optional. URL. Maximum length: 100 - url: String - Optional. Standard values are HOME and WORK. Maximum length: 1000
template	Object Conditional within <code>whatsapp</code> object when <code>type= "template"</code>. This type is used for sending messages based on approved templates for a given WhatsApp business account. Templates are viewable through Meta business manager as well as Messaging platform portal. For adding, editing, or deleting templates contact Messaging platform Support. Includes below parameters: • name: String

	Mandatory within template object, represents the name and identifier of the used template as named on Meta business manager and the Messaging portal • <code>language</code>: String. Mandatory within template, represents the code of the language or locale to use. Accepts either language and language_locale formats (e.g., en and en_US). Maximum length 5 characters • <code>header</code>: Object Optional. Represents header object of template message - <code>type</code>: String Mandatory. Represents type of header for the template: - <code>parameters</code>: Array of strings Optional. Maximum 1. To replace variable placeholder if found in a text. template header - <code>image</code>: Object Conditional within header object if template type is set to image. And type="image" ▪ <code>url</code>: String Mandatory within <code>image</code> object, represents the link to the image to be shared with WhatsApp receiver as part of interactive header. Supports HTTP/HTTPS URLs. ▪ <code>caption</code>: String Optional within <code>image</code> object. Describes the specified image for WhatsApp message receiver within interactive message header - <code>document</code>: Object Conditional within <code>header</code> object when type="document" in interactive message header. Fields include: ▪ <code>url</code>: String Mandatory within document object, represents the link to the document to be shared with WhatsApp receiver as part of interactive message header. Use only with HTTP/HTTPS URLs. ▪ <code>caption</code>: String
--	---

	Optional within document object within header of an interactive message. Describes the specified document for WhatsApp message receiver within interactive message header - video: Object Conditional within <code>header</code> object when <code>type="video"</code> for an interactive message ▪ url: String Mandatory within video object, represents the link to the video to be shared with WhatsApp receiver within an interactive message header. Supports HTTP/HTTPS URLs only ▪ caption: String Optional within video object. Describes the specified video for WhatsApp message receiver represents the link to the video to be shared with WhatsApp receiver within an interactive message header • body: Object Conditional, includes array of values, to replace variable placeholder in template's message body. - parameters: Array of strings Conditional, Maximum 32,768 characters if no header or buttons included, and 1024 in case button is included • buttons - type: String Mandatory for setting button type, should match type set on message template. Available types are: <code>quick_reply</code>, <code>url</code> and <code>phone_number</code>. - postbackData: String Conditional when <code>type="quick_reply"</code>. Represents the data returned on webhook URL when WhatsApp message receiver clicks a button. Should be unique value within same message. - parameters: Array of strings Conditional. Values replace variable placeholder within the template message body, according to order in the array (i.e. order should match the placeholder index)
--	---

Example Connect API Requests

SMS Request

POST enterprise.messagingsuite.smart.com.ph/cgpapi/messages/sms
Authorization: Basic abcdef1234567890==

Content-Type: application/json;charset=UTF-8
Content-Length: 358

```
{
  "messageType": "sms",
  "destination": "61299002200",
  "text": "Hello Connect API"
}
```

Note: If working with SMS Broadcast templates (Text Only) that have Subscribe / Un-Subscribe links present, Users are requested to provide the same template name as saved within the SMART Messaging portal.

```
{
  "messageType": "sms",
  "destination": "61299002200",
  "templateName": "Sample Sub/Un-Sub SMS Template"
}
```

RESPONSE:

HTTP/1.1 201 Created
Location: enterprise.messagingsuite.smart.com.ph/cgpapi/messages/sms/132211
Content-Length: 0

When Fraud Detection feature (parameter = fraudCheck) is enabled :

REQUEST :

```
{
  "messageType" : "sms",
  "destination": "61400000000",
  "text": "Your PIN for bank transfer is 1234",
  "fraudCheck" : true,
  "riskThreshold": 85
}
```

RESPONSE:

```
{
  "id" : 2081027,
  "msisdn" : "61299002200",
  "fraudulent" : true
}
```

To read more on Fraud Detection, Company Admins may refer to the Company Setup sections within Soprano Online Help sections.

Email Request

POST enterprise.messagingsuite.smart.com.ph/cgpapi/messages/email

Authorization: Basic abcdef1234567890==
Content-Type: application/json;charset=UTF-8
Content-Length: 358

```
{
  "messageType": "email",
  "destination": "somebody@somedomain.com",
  "templateName": "my_email_promo",
  "keyValues": "Product=Connect API;Company=My Compnay",
  "source": "myemail.promo@mydomain.com"
}
```

Note: If working with Email templates (HTML Only) that have Subscribe / Un-Subscribe links present, Users are requested to provide the same template name as saved within the SMART Messaging portal.

RESPONSE:

HTTP/1.1 201 Created
Location: enterprise.messagingsuite.smart.com.ph/cgpapi/messages/sms/132212
Content-Length: 0

Connect API (Email) possesses the capability to allow Users to attach PDF attachments to the Email Orders.

A new object array has been added to the API. For example ; inside JSON will see this if attachments are present:

```
"files" : [
{ "name" : "invoice1.pdf", "contentType" : "application/pdf", "data" :
"BASE64string1"},
{ "name" : "invoice2.pdf", "contentType" : "application/pdf", "data" :
"BASE64string2"}
]
```

In terms of attachment types, if templates have 2 files attached and 3 files as direct attachments, the email is sent with a total of 5 attachment files. Customers may configure some static file names for all files (For example ; welcome.pdf) within the Templates, and personalized pdf(s) inside the attachment files.

RCS Request

POST enterprise.messagingsuite.smart.com.ph/cgpapi/messages/rcs

Authorization: Basic abcdef1234567890==
Content-Type: application/json;charset=UTF-8

Content-Length: 358

```
{
  "messageType": "rcs",
  "destination": "61299002200",
  "templateName": "my_rcs_reminder",
  "keyValues": "Name=Mary;OTP=123456",
  "clientMessageId": "ABC-3-123",
  "costCentre": "XYZ123",
  "replyToTON": 10,
  "replyTo": "https://myweb.com/callback",
  "launchDateTimeStamp": "2020-05-23T04:25:00.000Z",
  "valid": 120,
  "fallbackMessageType": "sms",
  "fallbackTemplateName": "my_fallback_sms"
}
```

RESPONSE:

HTTP/1.1 201 Created

Location: enterprise.messagingsuite.smart.com.ph/[cgpapi/messages/sms/132213](https://enterprise.messagingsuite.smart.com.ph/cgpapi/messages/sms/132213)

Content-Length: 0

In the example above, if the template content was "Hello {Name}, your one-time-password is {OTP}.", the message received on the handset appears as "Hello Mary, your one-time-password is 123456.". If the RCS message was not delivered, then the message will automatically fallback to SMS, using the template 'my_fallback_sms'.

WhatsApp Request

POST enterprise.messagingsuite.smart.com.ph/[cgpapi/messages/whatsapp](#)
HTTP/1.1

Authorization: Basic YWxpYXNAc29wcmFuby5jb20uYXU6cGFzc3dvcmQ=

Sample JSON payload for sending a message:

```
{
  "destination": "61491089858",
  "messageType": "whatsapp",
  "clientMessageId": "XX-999.3d529a31-aac1-4446-a02a-1ffef6ab3501.00000000984.000003215",
  "templateName": "Alert_XX-999ZAM032_whatsapp",
  "launchDateTimeStamp": "2017-02-14T07:06:24.308Z",
  "validityDateTimeStamp": "2017-02-14T07:06:24.308Z",
  "keyValues": "Name=John;Balance=10.00"
}
```

RESPONSE

HTTP/1.x 201 Created

Location:

enterprise.messagingsuite.smart.com.ph/[cgpapi/messages/whatsapp/<location_id>](#)

Content-Type: application/json

Note: Here, <location_id> refers to the <message_d>.

In case of any error, standard 4xx and 5xx HTTP status codes will be returned.

- **4xx:** Client Side Error - Indicates that troubleshooting is needed on the client side. The error message with the code will indicate further details.
- **5xx:** Server Side Error - This error could be temporary or permanent. It is recommended to try again and if the problem persists, contact the Account Admin or the Support Team.

Example:

HTTP/1.x 400 Bad Request

Content-Type: application/json

```
{
  "statusCode" : 400,
  "statusText" : "Bad Request",
  "errorType" : "CLIENT_ERROR",
  "errorDescription" : "Message validity is in the past"
}
```

Message Samples for WhatsApp messages, when the WhatsApp parameter is being used with additional WhatsApp message types:

Text Message Sample

Request:

```
{
  "destination": "61491089858",
  "messageType": "whatsapp",
  "whatsapp": {
    "type": "text",
    "text": {
      "previewUrl": true,
      "body": "Test message for a text WhatsApp Message"
    }
  }
}
```

Response:

```
{
  "id": 7469666,
  "destination": "61491089858",
  "status": "ENROUTE"
}
```

Preview false:- just change "previewUrl": false,

Interactive Message List Samples**Request**

```
{
  "destination": "61491089858",
  "messageType": "whatsapp",
  "whatsapp": {
    "type": "interactive",
    "interactive": {
      "type": "list",
      "header": {
        "type": "text",
        "imageNo": {
          "type": "image",
          "url": "https://test.domain.com/interactive/abc.jpg"
        }
      },
      "text": "Hellow"
    },
    "body": {
      "text": "how are you?"
    },
    "footer": {
      "text": "See you"
    },
    "action": {
      "name": "menu",
      "sections": [
        {
          "title": "Test Message",
          "rows": [
            {
              "postbackData": "Test Postback",
              "title": "Test",
              "description": "This is a test"
            },
            {
              "postbackData": "Postback",
              "title": "Test2",
              "description": "this is another test"
            }
          ]
        }
      ]
    }
  }
}
```

```

    }],
    {
      "title": "Test2",
      "rows": [
        {
          "postbackData": "Postback2",
          "title": "Testing",
          "description": "This is another testing"
        }
      ]
    }
  ]
}

```

Response:

```

{
  "id": 7469671,
  "destination": "61491089858",
  "status": "ENROUTE"
}

```

Location Message Sample**Request:**

```

{
  "destination": "61491089858",
  "messageType": "whatsapp",
  "whatsapp": {
    "type": "location",
    "location": {
      "longitude": -33.838353585320284,
      "latitude": 51.5005765,
      "name": "LocationName Test",
      "address": "231, X Avenue, Santa Paolo, BR"
    }
  }
}

```

Response:

```

{
  "id": 7469674,
  "destination": "61491089858",
  "status": "ENROUTE"
}

```

Media Message Sample**Request**

```

{
  "destination": "61491089858",
  "messageType": "whatsapp",
  "whatsapp": {
    "type": "audio",
    "audio": {
      "url": "https://subdomain.domain.com/media/sooundfile.mp3"
    }
  }
}

```

Response:-

```

{
  "id": 7469677,
  "destination": "61491089858",
  "status": "ENROUTE"
}

```

Template Message Sample

Request:

```
{
  "destination": "61491089858",
  "messageType": "whatsapp",
  "whatsapp": {
    "type": "template",
    "template": {
      "name": "New_Template",
      "language": "es",
      "header": {
        "type": "image",
        "image": {
          "url": "https://subd.domain.com/media/image.jpg"
        }
      },
      "body": {
        "parameters": [
          "SampleImage"
        ]
      }
    }
  }
}
```

Response:-

```
{
  "id": 7469683,
  "destination": "61491089858",
  "status": "ENROUTE"
}
```

GBM Request

POST enterprise.messagingsuite.smart.com.ph/cgpapi/messages/businessmessages

HTTP/1.1

Authorization: Basic YWxpYXNAc29wcmFuby5jb20uYXU6cGFzc3dvcmQ=

Note: destination parameter here refers to the UUID that represents the destination.

Sample JSON payload for sending a message: Single Destination Based

```
{
  "destination": "0e4c7249-5792-4fe3-b06f-a39ae2744369",
  "messageType": "businessmessages",
  "clientMessageId": "XX-999.3d529a31-aac1-4446-a02a-1ffef6ab3501.00000000984.000003215",
  "templateName": "Test_GBM_API_Template",
  "launchDateTimestamp": "2017-02-14T07:06:24.308Z",
  "validityDateTimestamp": "2017-02-14T07:06:24.308Z",
  "keyValues": "Name=John;Balance=10.00"
}
```

Sample JSON payload for sending a message: Multiple Destination Based

```
{
  "destination": "0e4c7249-5792-4fe3-b06f-a39ae2744369",
  "messageType": "businessmessages",
  "clientMessageId": "XX-999.3d529a31-aac1-4446-a02a-1ffef6ab3501.00000000984.000003215",
  "templateName": "Test_GBM_API_Template",
  "source": "c508e5bd-5011-402c-8f6f-753fe373cd0c",
  "launchDateTimestamp": "2017-02-14T07:06:24.308Z",
  "validityDateTimestamp": "2017-02-14T07:06:24.308Z",
  "keyValues": "Name=John;Balance=10.00"
}
```

RESPONSE

HTTP/1.x 201 Created

Location:

enterprise.messagingsuite.smart.com.ph/[cgpapi/messages/businessmessages/<location_id>](#)

Content-Type: application/json

Note: Here, <location_id> refers to the <message_id>.

In case of any error, standard 4xx and 5xx HTTP status codes will be returned.

- 4xx: Client Side Error - Indicates that troubleshooting is needed on the client side. The error message with the code will indicate further details.
- 5xx: Server Side Error - This error could be temporary or permanent. It is recommended to try again and if the problem persists, contact the Account Admin or the Support Team.

Example:

HTTP/1.1 201 Created

Date: Fri, 01 Jul 2022 05:04:35 GMT

Content-Type: application/json

Content-Length: 246

Connection: keep-alive

Keep-Alive: timeout=99

Set-Cookie: JSESSIONID=1363952946405275315; Path=/; secure; HttpOnly

Location:

<https://qa1.devops.sopranodesign.com/cgpapi/messages/businessmessages/7321579>

X-debug-message: 20181205204458

X-Frame-Options: SAMEORIGIN

Access-Control-Allow-Origin: *

Access-Control-Allow-Methods: GET, POST, OPTIONS

Access-Control-Allow-Headers: Content-Type

X-Content-Type-Options: nosniff

X-XSS-Protection: 1; mode=block

Strict-Transport-Security: max-age=31536000; includeSubdomains;

Vary: User-Agent

```
{
  "id" : 7321579,
  "source" : "c508e5bd-5011-402c-8f6f-753fe373cd0c",
  "messageType" : "businessmessages",
  "destination" : "0e4c7249-5792-4fe3-b06f-a39ae2744369-ENROUTE,67531af8-3390-4135-a7e8-6ca0580c10ec-ENROUTE",
  "status" : "ENROUTE"
}
```

```
}
```

Push Notifications

POST enterprise.messagingsuite.smart.com.ph/cgpapi/messages/pushnotification

Authorization: Basic abcdef1234567890==

Content-Type: application/json;charset=UTF-8

Content-Length: 358

```
{
  "destination":
    "c4Ec_ZC8SCCF6dUWRg9lWz:APA91bHqpWVnS3xQqRXzGWXxFEwYjgI4I7DqoMqtl_MpTAKwFCYhS2BwGbDO
    o6GjVe1LXn7khnu3vVGfclYuwAgi8FBDTnEiEXSFlmlmDnuMWU3_iIaYz42rIQJXv6aN0blaDibJlmgJ",
  "messageType": "pushnotification",
  "source": "demoApp",
  "fallback":{
    "templateName":"orderPush",
    "messageType":"sms",
    "destination":"614999999999"
  },
  "pushNotification": {
    "notification": {
      "title": "This is a sample title!",
      "body": "This is a sample body for push notifications",
      "image": "https://mediasample.com/images/sample.jpg"
    },
    "android": {
      "clickAction":"clickAction",
      "channelId":"channelId",
      "isBigpicture":true,
      "sound":"sound"
    },
    "apns": {
      "category":"category",
      "isContentAvailable":true,
      "targetContentId":"targetContentId",
      "sound":"sound"
    },
    "data": {
      "actionButton1":"button1",
      "actionButton2":"button2",
      "actionButton3":"button3"
    }
  }
}
```

RESPONSE

HTTP/1.x 201 Created

Location:

enterprise.messagingsuite.smart.com.ph/cgpapi/messages/pushnotification/123456789

Content-Type: application/json

In case of any error, standard 4xx and 5xx HTTP status codes will be returned.

- 4xx: Client Side Error - Indicates that troubleshooting is needed on the client side. The error message with the code will indicate further details.
- 5xx: Server Side Error - This error could be temporary or permanent. It is recommended to try again and if the problem persists, contact the Account Admin or the Support Team.

Receiving Messages

Messages are received via webhook, that the client has implemented to listen to incoming events via parameters.

Incoming events can be user-response messages or message-status notifications.

Webhook HTTP headers should be set to Content-Type: application/x-www-form-urlencoded

Retry on forwarding webhook URL

If the client's webserver doesn't respond when the webhook is forwarded to their server, a retry will occur every 20 minutes for a 24 hour period.

Receive Message body

The following is a list of parameters are included in the HTTP GET request to client's webhook response, and specific to replyToTon=10 which returns the responses back to a HTTPS URL. For replyToTon=12 which is meant to deliver messages in JSON, please refer to the Soprano Connect YAML.

Parameter	Description
messageId	Integer The unique ID of this inbound message Example: 1047481234
source	String Sender Address i.e. MSISDN in E.164 format
description	String Service address that sender sent the message to
text	String The text message as typed by the user or status notification text as provided by the network

Parameter	Description
clientMessageld	String Client Reference as provided within the original outbound message Example: Card:4324
costCentre	String Cost Centre as provided within the original outbound message Example: SUPPORT
networkSubmitTime	String When event is received from network, in format yyyy-MM-dd HH:mm:ss in customer's timezone Format: date-time
clientSubmitTime	String When event is delivered to webhook, in format yyyy-MM-dd HH:mm:ss in customer's timezone" Format: date-time
segmentNumber:	Integer Part number only if received inbound user-message is a concatenated SMS Example: 1 Note: Only applicable to message type : SMS
segmentCount	Integer Total number of parts only if received inbound user message is a concatenated SMS Example: 4 Note: Only applicable to message type : SMS

Parameter	Description
referenceNumber	Integer Reference number only if received inbound user message is a concatenated SMS Example : 135 Note: Only applicable to message type : SMS
mtMessageId	Integer Status Notification: Message ID for this event
mtStatus	String Enum [DELIVERED, READ, EXPIRED, UNDELIVERABLE, REJECTED, DELETED, ACCEPTED, UNKNOWN] Status Notification: Last Status of the outbound message Example: DELIVERED
mtDoneDate	String Status Notification: Done Date as provided by the network as yyyy-MM-dd HH:mm:ss in customer's timezone Format: date-time
mtSegmentNumber	Integer Status Notification: Part Number if this event relates to lengthy SMS that is sent Example: 2 Note: Only applicable to message type : SMS
mtTotalSegments	Integer Status Notification: Total Number of Parts if this event relates to a lengthy SMS that is sent Example: 5

Parameter	Description
	Note: Only applicable to message type : SMS

Creating Templates

One of the advantages about the Connect API is that developers don't have to worry about building message content. All the content can be built in the portal without any need for coding.

When creating message content in the portal, all the messaging channels follow a similar format. The advantage being that they apply to all the channels, SMS, RCS, WhatsApp and EMAIL.

Within the SMART Messaging a message for any channel is known as an **order**. This order can be saved as a **template**. The **template** created is used by the Connect API to send the message to the recipient.

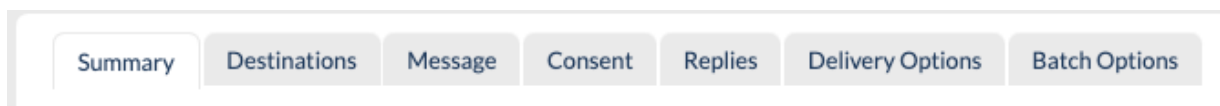
A template must be created and owned by the Connect API user associated with the account (parent or child account), otherwise the template will not be found.

When the Connect API user specifies a templateName, the search results are dependent on the following search criteria:

- Parent Level searches are done first as the message is sent by the parent level user, in case of multiple results, 'last modified at' will be compared and the template with the most recent value selected.
- If the template is not found at the parent level, then the search will be done on any child accounts that exist. In case of multiple results, 'last modified at' will be compared and the template with the most recent value selected.
- Search is case-sensitive. It is strongly recommended not to have multiple templates with identical names.

All templates must be created in the same customer account of the Connect API user, that is created in the parent or child account. Any template outside of this customer record will not be found.

When creating content, all message types will have the below tabs. Each of these tabs has a specific setting that Connect API uses. Below highlights the important ones:



TAB	SETTING	CONDITION	DESCRIPTION
Summary	Name	Mandatory	Name of the template this will be used by the Connect API to send the message content.
	Labels	Optional	Optional ability to tag messages. Requires an additional license.
	Description	Optional	Optional description of the message template.
Destinations	Destination	Not Used	This is not required. Any destination set in the Template will be overridden by the Connect API request destination parameter.
Message	Message	Mandatory	Channel specific content is created here. Merge variables like {Balance} or {1} can be added in text fields when building content. They will be passed as keyValues by the Connect API.
Consent Tab	Consent	Optional	Set the Outbound Filter Type to BLACKLIST/WHITELIST and select 'Outbound Filter List' that contains the blacklist or whitelist. Messages will be filtered based on blacklist/whitelist settings.
Delivery Options	Registered	Optional	Applicable only when messageType=sms. Setting the registered flag here means it does not need to be set in every API request. All other messageType have registered set by default.
Delivery Options	Source Address	Optional	For message type SMS and Email, specify the sent message's "source" address by setting the Source Address. Note 1: If you specify the 'source' parameter in the API request, this setting in the order template will be ignored. Note 2: If the Connect API User is set up in the parent account, any SMS or Email Order Templates in sub-accounts should have their 'Source Address' set to 'NONE'. The 'source' address to be used should be set in the Connect API request.
Delivery Options	Fallback Channel	Optional	Only applicable when messageType=rcs. SMS is the only option to select. This will enable automatic SMS fallback to SMS if an RCS message can't be delivered. Same applies to WhatsApp.
Delivery Options	Fallback Template	Optional	If a Fallback Channel is selected then it is possible to select the Fallback Template. Currently only SMS is supported. If RCS or

TAB	SETTING	CONDITION	DESCRIPTION
			WhatsApp message fails to deliver, then this SMS Template will used. Fallback SMS supports both 'Text' and 'HTML' content types.

Connect API HTTP Batch Mode

Connect API HTTP in batch mode

The following sample represent the array of messages about message submission for all applicable channels:

Sample request:

POST /cgpapi/batch/messages

```
[
  {
    "messageType": "sms",
    "source": "112233",
    "destination": "61299002201",
    "text": "hi Mary, your promo code is 123456"
  },
  {
    "messageType": "sms",
    "source": "Promo",
    "destination": "61299002202",
    "text": "hi {name} your balance is below {balance}",
    "keyValues": "name=Peter;balance=456"
  },
  {
    "messageType": "rcs",
    "source": "Reminder",
    "destination": "61299002203",
    "templateName": "Balance_Reminder",
    "keyValues": "due=1000;currency=AUD;date=2022-Nov-29"
  }
]
```

Sample response:

200 OK

```
[
  {
    "id": 4638793,
    "messageType": "sms",
    "destination": "61299002201",
    "status": "ENROUTE"
  },
  {
    "id": 4638794,
    "messageType": "sms",
    "destination": "61299002202",

```

```
"status" : "ENROUTE"  
},  
{  
  "id": 4638797,  
  "messageType": "rcs",  
  "destination": "61415000003",  
  "status": "ENROUTE"  
}  
]
```

Connect API SFTP Batch Mode

Connect API SFTP in batch mode could actually be considered a no-code option. Although to automate the solution, it would require extracting customer's contact data and creating a CSV list file that contains the API parameters to execute, if tightly integrated SMART's API parameters into your application. No changes are needed to the application. Send the file in the correct format into an SFTP server. It will be picked up in SMART Messaging and execute all the Connect API parameters.

Create a CSV file that conforms to [RFC4180](#) and the file should be saved as UTF-8.

The CSV file requires a header line having the API parameters defined in the Connect API real-time mode. This includes the mandatory fields used to add any optional fields.

Example header file:

destination,messageType,templateName,keyValues,clientMessageld

For each destination, add a row in the CSV list file with the values required for each field. The file can contain multiple messageType. Also, this can target different templates with the same file. This enables multiple ways to send customer messages in a single batch file.

Example data for sending an SMS message:

destination,messageType,templateName,keyValues,clientMessageld
61299002200,sms,One Time Password,OTP=34252,login#2345

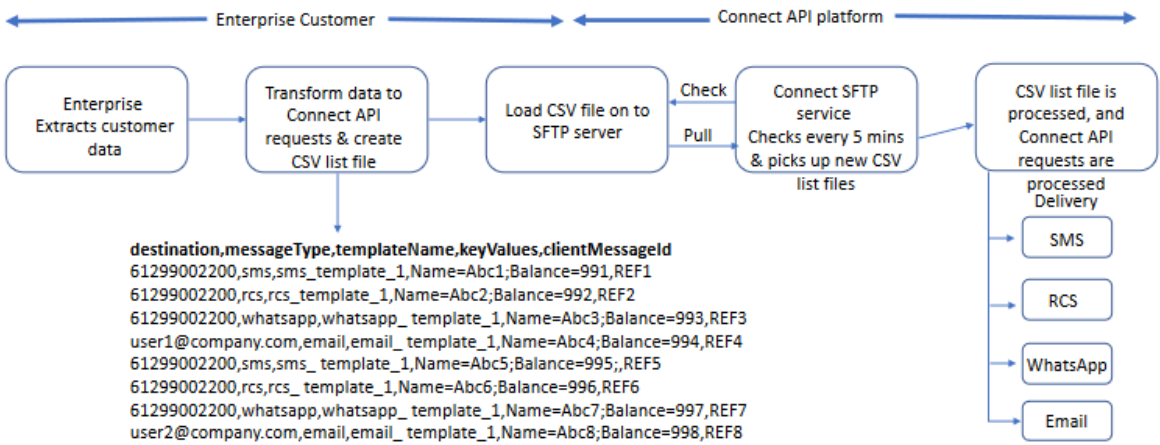
Send to multiple destinations with different a messageType. templateName, keyValues to provide personalized messages on any supported channel in the same CSV list file.

Destination,messageType,templateName,keyValues,clientMessageld

61299002200,sms,sms_template_1,Name=Abc1;Balance=991,REF1
61299002200,rcs,rcs_template_1,Name=Abc2;Balance=992,REF2
61299002200,whatsapp,whatsapp_template_1,Name=Abc3;Balance=993,REF3
user1@company.com,email,email_template_1,Name=Abc4;Balance=994,REF4
61299002200,sms,sms_template_1,Name=Abc5;Balance=995,,REF5
61299002200,rcs,rcs_template_1,Name=Abc6;Balance=996,REF6
61299002200,whatsapp,whatsapp_template_1,Name=Abc7;Balance=997,REF7
user2@company.com,email,email_template_1,Name=Abc8;Balance=998,REF8

The diagram below shows the flow from the enterprise to being processed by the Connect API service.

Connect API – Bulk Mode via SFTP



Create Lists in The Portal

In order to automate the processing of CSV files processed by the SFTP service, create a list. There are some specific settings are required to be setup in the portal:

Navigate to **Home -> Create a New List -> Owner Type -> Shared**

Tab	Setting	Condition	Description
List Information	Name	Mandatory	Unique List Name - this will be the AUTOMATIC API parent list.
	Description	Optional	Short description for list use
	List Presentation	Optional	Provides a way to insert additional information from the CSV file that is not sent with the message. Will appear in the message reports. Use the following: Plain text List Presentation fields either {name} when name is defined in the column header or {n} where n is the column number.
Content	Method	Mandatory	Select AUTOMATIC_API
	Retrieval Frequency	Mandatory	For Example ; Select EVERY 5 minutes
	Start Date	Mandatory	Set the start date for the list processing to start, normally set to today's date
	Start Time	Mandatory	Has to be set in the future from the current time. Set about 5 minutes from the current time to complete the other setting
	Protocol	Mandatory	SFTP is the default and can't be changed for AUTOMATIC API
	URL	Mandatory	URL location for the SFTP server with following formats supported: Mydomain.com Mydomain.com:port e.g. Mydomain.com:22 192.168.1.1 192.168.1.1:22 Specify multiple URL's separated by a new line, if picked up from multiple servers like primary and secondary.
	Username	Mandatory	SFTP username
	Password	Optional	SFTP password if required
	FTP File Name	Mandatory	Various CSV filenames can include wildcard (For Example Myfile*.csv) a path can be included which can be absolution.

Tab	Setting	Condition	Description
			(sample /mydir/absolute/myfile*.csv) or relative (sample relative/one/myfile*.csv)
	Source File Encoding	Mandatory	For Example ; Select UTF-8
	Rename Once Collected	Optional	We recommend that the file is renamed. Once collected select RENAME to rename the file with '.collected' extension. Once collected select MOVE if file is to be moved into the processed directory and/or error sub-directories. This setting is useful to process failed records out from master data files.
	Ignore Duplicate Files	Optional	This should be checked if using multiple SFTP servers and remember to place the same file on each server.
	Suppress Email Notifications	Optional	Email Notifications are sent on upload completion and /or when errors occur in the list processing. Default action being - send to list owner.
	Notification List	Optional	Create a list that contains people that want to be notified. The list can contain email addresses and mobile numbers.
	With Pipe Delimiter	Optional	Generally, .CSV files are separated by commas, however sometime files (especially from older systems) may be separated by Pipe Delimiter ' ' .
	With Footer Line	Optional	Some CSV files have a footer line. Setting this enables it to be skipped so that it does not get discarded as an error.
	Ignore Duplicate Records	Optional	A duplicate record is where the mobile number and the final content after mailmerge are repeated 2 or more times.
	File/Contents Starts with	Mandatory	Select HEADER LINE as uploaded. The .CSV file should contain a header file.

Fraud Detection and Prevention within Connect API

Introduction

Fraud Detection is the system's ability to detect any malicious activities that may compromise the system's security protocols. This may be in the form of (for example) phishing customer's personal details. In this

particular case, looking at the situation where Fraudsters are known to harvest personal customer information through social media, or in some cases purchase the data direct from areas such as dark web. Using this information, it is possible for them move a number to a device or SIM card under their control, without the real customer's knowledge. There are growing numbers of cases where fraudsters can infect a smart-phone with a virus that setup a divert, forwarding all calls and messages to a different number with the victim's knowledge.

When these events occur it exposes your organization to risk and your customers to fraud.

A dedicated Fraud Detection API / endpoint is present so user can query to get fraud risk scores in JSON format.

Note: Fraud Detection and Prevention is a licensed feature.
Users must procure appropriate licenses to access it's features.
reach out to <%SALES_URL%> for more details.

Fraud Detection Setup in MEMS



The Fraud Detection and Prevention section within Company Setup serves a very special purpose. This feature prepares your SMART Messaging portal to detect and alert you of specific security threats.

Company Setup > Customer Information > Fraud Detection and Prevention Preference

The fraud in question here refers to SIM-swap (data theft), SS7 vulnerability (network manipulation), SMS Malware (data theft) and Smishing (identity theft).

Configuration



When Users click on the Fraud detection section within Company Setup, the above screen is presented. Admins can take advantage of the various field in this section to configure SMART Messaging portal to capture any of the previously mentioned threats.
Here are the fields in details below:

SIM Swap Weightage

The level of priority SMART Messaging gives to prioritize SIM Swap checks and detects if a possible SIM Swap scenario has occurred. Choose a value from 0 - 100. 0 meaning SMART Messaging ignores SIM-swap functionality.

SIM Swap Date Score

Once the overall SIM Swap weightage has been assigned, the Admin can assign Weightage split based on a different time gap ranges.

Last 24 Hours

Score allocated to the system to detect if SIM Swap has occurred in the last 24 hours. Value can be 100 to actively check for SIM Swaps every 24 hours

7 to 1 days ago

Score allocated to the system to detect if SIM Swap has occurred between a gap of one day to seven days (one week). Weightage could be slightly less compared to Daily checks (for example assign values as 80)

14 to 7 days ago

Score allocated to the system to detect if SIM Swap has occurred between a gap of seven days (one week) to fourteen days (two weeks). Weightage could be slightly less compared to one day to seven day (one week) checks (for example assign values as 60)

30 to 14 days ago

Score allocated to the system to detect if SIM Swap has occurred between a gap of fourteen days (two weeks) to thirty days (~ one month). Weightage could be slightly less compared to seven days (one week) to fourteen days (two weeks) checks (for example assign values as 40)

More than 30 days ago

Score allocated to the system to detect if SIM Swap has occurred more than thirty days (one month) ago. Weightage could be slightly less compared to fourteen days (two weeks) to thirty days (~ one month) checks (for example assign values as 20)

Call Forward Weightage	This is to have a check in place to detect of a customer call is being directed away from a trusted party towards someone with malicious intent. Assign a score between 0 - 100. 0 indicating no call-forwarding check is required.
Trusted Network Weightage	This is to have a check in place to detect of the Network being used is a Trusted source. Assign a score between 0 - 100. 0 indicating no Network check is required.
Total Risk Score	There is a specific calculation that occurs to produce the exact value of the overall Total Risk Score to the SMART Messaging portal.

Score Calculation : (SIM Swapped x Weightage x Date Score) + (Call Diverted x Weightage) + (Not Trusted Network x Weightage). For Example; If weightages are : SIM Swap = 60, Call Forward = 20, Trusted Network = 20, and SIM Swap Date Score result is: SIM swapped 3 days ago and both call divert activated and network are trusted, the total score would be $1 \times 60\% \text{ of } 80 + (1 \times 20) + (0 \times 20) = 68$

Default Risk Threshold	The Default risk threshold value if as such is not provided inside message API requests. If calculated risk score is higher then threshold value, message is taken as fraudulent and is not sent.
-------------------------------	---

Risk scores may be seen as:
 ≤ 20 = Negligible
 ≤ 40 = Low
 ≤ 60 = Moderate
 ≤ 80 = High
 ≤ 100 = Extreme

Customer	The name of the Customer handling the SMART Messaging portal.
ID	A unique value assigned to the Fraud Detection profile created.
Created By	Name of the Admin that created the Fraud Detection profile.
Last Modified By	Name of the Admin that last modified the Fraud Detection profile.
Created At	Date when the Fraud Detection profile was created.
Last Modified At	Date when the Fraud Detection profile was last modified.

Fraud Detection Parameters

Here are the main parameters that Users will work with when configuring Fraud Detection in the API.

Request Parameters

Parameter	Description	Example Values
msisdn	Numerical values. Dedicated mobile number that is involved in Fraud Check.	61432*****
riskThreshold	Numerical value. Optional. min=1, max=100	67

Response Parameters

Parameter	Description	Example Values
fraudulent	Boolean value Value is returned as part of the response. Values are TRUE or FALSE.	FALSE
riskScore	Numerical value. Optional. min=1, max=100	85
simSwapped	Boolean value Value is returned as part of the response. Values are TRUE or FALSE. Note: This value is determined by the SIM Swapped weightage values set in the Company Setup - Customer Information Preferences - Fraud Detection and Prevention Preferences (SMART Messaging Portal).	TRUE
callForwardActive	Boolean value Value is returned as part of the response. Values are TRUE or FALSE. Note: This value is determined by the Call Forward weightage values set in the Company Setup - Customer Information Preferences - Fraud Detection and Prevention Preferences (SMART Messaging Portal).	FALSE
trustedNetwork	Boolean value Value is returned as part of the response. Values are TRUE or FALSE. Note: This value is determined by the Trusted network weightage values set in the Company Setup - Customer Information Preferences - Fraud Detection and Prevention Preferences (SMART Messaging Portal).	FALSE

Once Users have grasped this concept it is easier to understand and apply these parameters to the HTTP API when performing Fraud Check and Prevention.

Advantages of Fraud Check and Prevention within Connect API

There are some advantages when Users opt to perform Fraud Detection and Prevention within the Connect API.

- Since Users already have access to the Connect API up and running, applying Fraud Detection and Prevention requires less effort.

Note: Users must still procure the dedicated Fraud Detection and Prevention API licenses.

- The usage of Fraud Detection and Prevention is easier with minimum changes to the API, when using it in tandem with Connect API, as compared to the standalone Fraud Detection and Prevention API.

Appendix

How to get access token

API authentication

All API requests need to be authenticated through the Authorization Bearer header. The authentication method is OAuth 2.0 Client Credentials flow in which an application is exchanging its application/API/client credentials for an access token over a separate endpoint.

This type of authentication is the most secure option and is almost the industry standard.

When an access token is returned in response, it will expire within the time limit given in the response, measured in seconds. It is important to note that once the token expires, a new token must be generated in order to continue using the service.

MEMS acts as both a resource and an authorization server, however it is possible you application to integrate with any external provider like Azure, Okta, etc, ask your Account Manager for more details if interested in such.

How to get client credentials

Customer Administrator of your account to login into portal and create them by navigating to Company Setup -> Manage Users -> API Keys -> New page

- Give some name for your orientation
- Link it to one User from your Account
- Save

From displayed summary page copy/paste API ID and Key, e.g.

API ID: 12345678

API Key: abcdefgh1234567890abcdefgh1234567890

You can create maximum two keys, and can rotate them at any convenient time.

How to use OAuth 2.0

Make a call to get the access token and the expiration time from a separate endpoint.

Include "Bearer " and the token in the Authorization header for all subsequent calls until the token expires.

Authorization: Bearer <access_token>

Create OAuth2 token Client Credential flow

Generate OAuth2 access token in order to authenticate all other API calls in this guide.

POST /rest/auth/token
application/x-www-form-urlencoded

Parameter	Description
client_id (Required)	String The API ID which was created by the user.
client_secret (Required)	string The API Key which was created by the user..
grant_type (Required)	string Value: "client_credentials" Grant type as defined by the OAuth framework

Connect API Status and Error Codes

This tells you the status of the request and the possible cause of error, if any. Here is the explanation and the standard list of codes that may help you in troubleshooting.

- **1xx: Informational** - Indicates a provisional response. There are no required headers for this class of status code.
- **2xx: Success** - Request was successful. The information content in the response depends on the request type.
- **3xx: Redirection** - Indicates that further actions needs to be taken by the user agent.
- **4xx: Client Side Error** - Indicates that troubleshooting is needed on the client side. The error message with the code will indicate further details.
- **5xx: Server Side Error** - This error could be temporary or permanent. It is recommended to try again and if the problem persists, contact the Account Admin or the Support Team.

Code	Message	Description
200	Success	Result was successful.
201	Created	Request was fulfilled and a new resource has been created.
204	No Content	Request was fulfilled but doesn't need to return an entity body.
400	Bad Request	Request was invalid or cannot be otherwise served. An accompanying error message will explain further.
401	Unauthorized	Authentication credentials are missing or incorrect.
404	Not Found	URI requested is invalid or the resource requested doesn't exist.
405	Method Not Allowed	Method doesn't exist or is not allowed.
409	Conflict	Request could not be completed due to conflict with the current state of the resource.
500	Internal Server Error	Our server is down. Please try again in sometime.
501	Not Implemented	Server doesn't support the functionality required to fulfill the request.

WhatsApp set Message as Read and MO Parameters

Set captured Message (MO) as read:

To set message as read (convert ticks to blue), add the specified message id to the endpoint URL ex: /77326734. In addition to one parameter in body:
status: String Mandatory. Allowed value: "read".

MO WhatsApp Parameters:

To setup a default webhook on specific URL to capture all incoming messages (MO) and delivery receipts for a given address (phone number), refer to Messaging platform admin sharing the URL on which the webhook should be setup.

Parameter	Description
accountId	Customer's Id.
statuses	Object Holds status of a message when WhatsApp Message receiver reads or receives message. Includes below Parameters: - channel: String Presents the channel on which message was captured (i.e. "whatsapp") - clientMessageId: String describes set client message id

	• costCentre: String Represents the cost centre • destination: String Represents WhatsApp phone number on which the webhook URL is set to capture all incoming messages from different WhatsApp users. Includes international prefix without the 00 ex: 61403xxxxxx • messageId: Integer Represents id of the message • id: Integer Represents id of the message ex: 77326734 • postbackData: String Represents postbackData set for the button/ list item clicked. • receivedAt: Datetime Represents date and time message was captured. ex:2022-08-02T07:39:56.815Z • source: String number of the message sender, includes International prefix without the 00 ex: 61403xxxxxx • whatsapp: Object. Conditional when <code>channel="whatsapp"</code> - type: String. Conditional when <code>event_type</code> is any of the following values: <code>text</code>, <code>button</code>, <code>contacts</code>, <code>location</code>. Represents type of the reply message sent by the receiver. - text: Object. Conditional when <code>eventType="text_reply"</code> or <code>"reaction_reply"</code>. Includes the following parameter: - o body: String. Includes text and emojis sent or reaction provided - button: Object shared when WhatsApp message receiver clicks a list item or a button - o buttonType: String. describes the button/ list item clicked by the message receiver. Values: <code>button_reply</code>, <code>list_reply</code>, <code>quick_reply</code>, - o description: String. Holds the description set for the list item clicked. - o postbackData: holds the post back data set for the clicked button - o title: holds the text displayed for the button/ list item clicked - media: Object. holds media shared as a reply by WhatsApp message receiver. Includes the following fields: - o caption: String. Holds the text that describes the media content - o category: String. Describes the category of media received. Values include: <code>image</code>, <code>document</code>, <code>audio</code>, <code>video</code> and <code>sticker</code> - o mimeType: String. Mime Type - o url: String. URL. - location: Object.
--	---

	Includes the below Parameters: - o longitude: Double Mandatory, represents longitude in degrees for the location. Ex:151.20982 - o latitude: Double Mandatory, represents latitude in degrees for the location. Ex: -33.83853 - o name: String Optional describes the name of location to be viewed by the receiver. Ex: North Sydney - o address: String Optional, represents address for location, will be only displayed if name field is present. Maximum 512 characters. - contacts: Array of objects. Includes the below Parameters: - o addresses an Array of Objects Optional within the "contacts" object. Holds the address(es) of the contact being shared on whatsapp. Includes the below fields: ▪ type: String. Optional describes the type of address. ex: Work. ▪ street: String. Optional, describes the unit, building number and street name. ex: 1 Blue Street. ▪ city: String. Optional describes the city name. ▪ state: String. Optional . Represents state abbreviation. Ex:NSW. Maximum length: 100. ▪ zip: String. Optional ZIP code. Ex "2060". ▪ country: String. Optional. Full country name "Australia". ▪ countryCode: String. Optional. 2 letter abbreviation. Ex:AU - o birthday: String. Optional YYYY-MM-DD formatted string. - o emails: Array of objects. Optional. Includes below parameters:
--	--

- **email:** String.
Optional. Email address.
- **type:** String.
Optional. Standard values are HOME and WORK.
- **name:** Object.
Mandatory. Includes below parameters:
 - **formattedName:** String.
Mandatory. Full name. Maximum length: 500
 - **firstName:** String.
Optional. First name. Maximum length: 100
 - **lastName:** String.
Optional. Last name. Maximum length: 100
 - **middleName:** String.
Optional. Middle name. Maximum length: 100
 - **suffix:** String.
Optional. Name suffix. Maximum length: 100
 - **prefix:** String.
Optional. Name prefix. Maximum length: 100
- **org:** Object
Optional. Contact organization information formatted as an org object. The object can contain the following field:

Contact organization information formatted as an org object. The object can contain the following fields:
 - **title:** String.
Optional. Contact's business title. Maximum length: 100
 - **department:** String.
Optional. Department name.
 - **company:** String.
Optional. Name of the contact's company.
- **phones:** Array of Objects.
Optional.
Contact phone number(s) formatted as a phone object. The object can contain the following fields:
 - **type:** string.
Optional. Automatically populated with the `wa\_id` value as a formatted phone number.
 - **phone:** String
Optional. Standard Values are CELL, MAIN, IPHONE, HOME, and WORK.
 - **waId:** string

	Optional. WhatsApp ID. o <code>urls</code> Array of objects. Contact URL(s) formatted as a <code>urls</code> object. The object can contain the following fields: - <code>type</code>: String - Optional. URL. Maximum length: 100 - <code>url</code>: String - Optional. Standard values are HOME and WORK.
--	---